Processing RAW images in Python

Working Paper · April 2017

DOI: 10.13140/RG.2.2.21522.25287

1 author:

P. Rojtberg
Fraunhofer Institute for Computer Graphics Research
23 PUBLICATIONS **227** CITATIONS

Processing RAW images in Python

Pavel Rojtberg

March 6, 2017

Abstract

This is a tutorial on reading and processing RAW photo files with Python. A RAW file stores unprocessed sensor readings. Therefore several conversion steps need to be applied before it can be displayed. Typically these steps are applied by the camera firmware or by a RAW development tool on the PC. In contrast this work presents a programmatic Python/NumPy based workflow allowing to easily modify the individual steps. Each required processing step is motivated and discussed.

1 Motivation and significance

With the recent focus on photo quality with phone cameras, raw sensor readings (RAW) start to be ubiquitously available. RAW data directly maps to the incident light captured by the device. Usually only one colour is recorded per pixel and the specific data format depends on the layout of the used sensor.

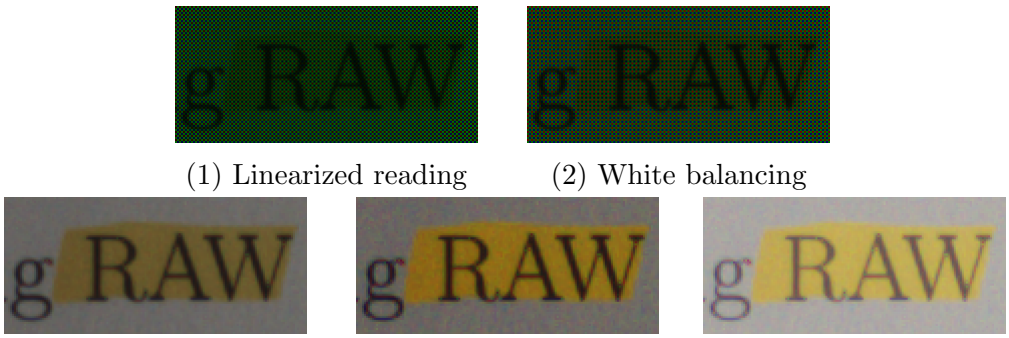
In contrast, the common representation of images is an array of RGB triples per pixel. This layout reflects typical display devices and allows mapping the stored values in a straightforward way to activation voltages for the device pixels. However much of the initially captured information is discarded that could otherwise be exploited by computer vision algorithms.

For instance most RGB image formats assume the sRGB [10] gamut for the display device, which only covers a subset of all visible colours. This assumption allows storing the colour values with only 8 bits of resolution. However typical sensors capture colours at about 14 bit resolution so using RAW data instead would greatly extend the dynamic range. Furthermore the

conversion to RGB images involves determining the illuminant which is then non-reversibly encoded in the pixel colour. Most vision algorithms rely on the pixel colour and local image contrast and therefore could directly exploit the extended value range of RAW data.

Additionally the specific structure of the captured RAW data can be explicitly considered in the algorithm which proved to be beneficial for super resolution [6], image compression [8] and denoising [16].

Although this indicates that other algorithms would also benefit from using RAW data, RGB images are still the prevalent data format for computer vision. One of the reasons is that until recently RAW data could only be captured with industrial or high-end Digital Single Lens Reflex (DSLR) cameras, which limited the possible application scenarios. Furthermore working with RAW data is more cumbersome as the data is not only sensor specific but also stored in a vendor dependent file format. Although there is a standardized file format for RAW data [1] and new system APIs [7] that allow an unified data access, this issue is still present.



(1) Linearized reading (2) White balancing

(3) Demosaicing (4) Colourspace conversion (5) Gamma correction

Figure 1: Processing steps of RAW developing

Next the data read from a RAW file cannot be used as is. Instead several processing steps (see figure 1) have to be applied before it can be displayed on a monitor or further processed.

This work therefore presents a workflow for reading RAW data and converting it to displayable RGB images that is also suitable for real-time processing. Given a specific use case the presented processing steps can be adapted, replaced and new steps can be introduced.

In contrast to the presented approach, existing RAW processing toolboxes like dcraw [3], the Adobe DNG Converter and the corresponding graphical

front-ends Lightroom and Rawtherapee only allow the parameterization of a fixed set of predefined steps. Plugging in a custom implementation of a specific step is not possible. Furthermore real-time processing of online recorded RAW data is prohibited by the design of the tools.

The workflow presented by Sumner [11] overcomes the former limitation. However it still requires an offline preprocessing step either using the Adobe DNG Converter (on Windows) or dcraw (otherwise). Furthermore it depends on the MATLAB environment. These choices hinder writing portable code and restrict the possible platforms/use-cases.

The presented workflow on the other hand only requires Python, NumPy the novel libraw.py[1] bindings. The decoding of vendor specific RAW files is done by means of libraw [12] library which internally uses the dcraw decoder. This way the proposed RAW processing can be easily integrated into a larger computer vision pipeline while retaining the high abstraction level comparable to that of the MATLAB environment. Furthermore the presented steps can be easily transferred to native code using the C++ API of libraw for better performance or in case the Python environment is not available. To our knowledge this is the first documented RAW processing workflow that works across all platforms while allowing real-time image processing.

This document is structured as follows: in section 2 the minimal required processing steps are presented and discussed. In section 3 the workflow is summarized and additional non-essential steps that can further improve the image quality are presented.

2 Converting RAW images to displayable RGB

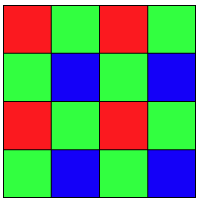


Figure 2: RGBG Bayer pattern

The starting point for RAW image conversion are the sensor readings. The sensor can be imagined as an array of probes that measure incident light. To record colours typically a colour filter is placed in front of the sensor such that each pixel only records the intensity of one primary colour — however there are also sensor technologies that record all primary colours per pixel [14].

The human visual system captures luminance in the green light spectrum [15], therefore colour filter arrays (CFA) or colour filter mosaics are designed to capture

[1] `https://github.com/paroj/libraw.py`

green light at a higher resolution. The most popular CFA arrangement is the Bayer Pattern (see Figure 2) which uses a filter for the red, green and blue colour primaries. However other patterns with different primaries (e.g. red, green, blue, white) also exist [13].

To abstract from the actual RAW file format (*.dng*, *.cr2*, etc.) the files are read using libraw. Alternatively the data could be obtained from a live image stream. From now on libraw is only needed to access the file metadata.

Listing 0:

```
import libraw                    # file decoding
import numpy as np               # processing

proc = libraw.LibRaw()           # create RAW processor
proc.open_file("file.dng") # open file
proc.unpack()                    # extract mosaic from file
mosaic = proc.imgdata.rawdata.raw_image
```

The decoded data is an array of 16 bit unsigned integers representing the sensor mosaic. The following processing steps need to be applied to obtain a displayable image:

1. Mapping to linear values
2. White balancing
3. Demosaicing
4. Colour space conversion
5. Gamma correction.

The remainder of this section discusses the individual steps in detail and presents possible processing methods. See Figure 1 for a visualization.

2.1 Mapping to Linear Values

In this step the sensor readings are transformed into linear reference values such that 0.0 encodes zero light and 1.0 encodes the saturation point of the sensor. For this we follow the DNG specification [1].

While the mosaic values linearly map to the incident light, the camera may encode them in a nonlinear fashion to improve storage precision. Therefore the first step is to apply a linearisation look up table (LUT) to get back

to linear values. In comparison this is a costly operation, therefore one should check whether the LUT is linear and thus the linearisation can be skipped.

Next we want to shift the values such that 0 encodes zero incident light. Typically sensors will read a value > 0 even in complete darkness due to the influence of ambient temperature and sensor noise. Most cameras therefore determine the black level by reading a hidden (i.e. not covered by the image circle) part of the sensor. Depending on the used camera this region is also included in the RAW data thus allowing to perform this step manually. However for convenience we will instead use the black level computed by the camera and stored in *imgdata.color.black*

Listing 1:

```
lin_lut = proc.imgdata.color.curve # linearization LUT
mosaic = lin_lut[mosaic]           # apply LUT

black = proc.imgdata.color.black
saturation = proc.imgdata.color.maximum
mosaic -= black                    # black subtraction

uint14_max = 2**14 - 1
mosaic *= int(uint14_max/(saturation - black))
mosaic = np.clip(mosaic,0,uint14_max)  # clip to range
```

The values can now be scaled such that the saturation point of the sensor moves to 1.0. To avoid copying we are going to keep the data in the integer representation. As the data is stored as 16 bit integers, the naive choice would be to move the saturation point to $uint16_{max} = 2^{16} - 1$. However the data must be scaled in the white balance step again — here values at $uint16_{max}$ would overflow. Furthermore the range of typical sensor cells is 10 - 14 bit [11]. Therefore we instead move the saturation point to $uint14_{max} = 2^{14} - 1$ which does not limit dynamic range while still allowing further scaling.

2.2 White Balancing

In the white balancing step the linear pixel values are multiplied such that image areas that are perceived as different shades of grey have equal component intensities (i.e. $R = G = B$). The point with the maximum pixel intensity is by definition called the white point. In theory only the ratios of

the multipliers matter as the actual values do not change the resulting white point. However this view disregards the effects of numerical clipping and sensor saturation. The colour filter in front of the sensor results in individual pixels being exposed to different amounts of energy. Statistically green pixels are saturated first [17] as green light is most common and has a higher photon energy than red light. Considering the integer representation of the image, also scaling the green channel would result in a reduction of dynamic range. Therefore we normalize the white balance multipliers such that the green multiplier becomes 1.0.

For obtaining the multipliers usually the illuminant needs to be estimated. For an overview of available algorithms see [5]. In the following we simply use the values provided by the camera in *imgdata.color.cam_mul*

Listing 2:

```
assert(proc.imgdata.idata.cdesc == b"RGBG")

cam_mul = proc.imgdata.color.cam_mul # RGB multipliers
cam_mul /= cam_mul[1]                # scale green to 1
mosaic[0::2, 0::2] *= cam_mul[0]     # scale reds
mosaic[1::2, 1::2] *= cam_mul[2]     # scale blues
mosaic = np.clip(mosaic,0,uint14_max)# clip to range
```

Here we assume the RGBG Bayer pattern. The normalization of the multipliers to green also minimizes the number of required multiplications; only 50% of the pixels (reds and blues) need to be scaled.

2.3 Demosaicing

When using a Bayer CFA (see Figure 2) for capturing, reds and blues are recorded only at a quarter of the image resolution while green is recorded at half of the image resolution. However as light intensity is still recorded at all pixels, a full resolution image can be reconstructed. This step is called demosaicing. Methods range from simple nearest-neighbour interpolation to sophisticated bayesian formulations (see [9] for a survey).

However if the goal is only to obtain a displayable RGB image a trivial solution can be obtained by downsampling. Instead of computing the RGB values at full resolution, we only compute the image at quarter resolution

— therefore we can simply use the red and blue channels and average the green channel. This is equivalent to demosaicing using nearest neighbour interpolation and then downsampling to quarter resolution.

Listing 3:

```
def downsample(m):
    r = m[0::2, 0::2]
    g = np.clip(m[0::2, 1::2]//2 + m[1::2, 0::2]//2,
                0, 2**14 - 1)
    b = m[1::2, 1::2]

    return np.dstack([r, g, b])

img = downsample(mosaic)          # displayable rgb image
```

Note that the *downsample* function above just illustrates the problem at hand. The image in figure 1.3 was created using the edge-aware demosaicing algorithm implemented in the OpenCV [2] library.

Domain specific adaptations

- **Computer vision** Further averaging the r, g, b components during down-sampling, one directly obtains a grayscale image with dynamic range of 14 bit. For image based tracking the loss of resolution is tolerable, while the extended dynamic range allows detection even in low-light situations.

- **Colour sciences** The demosaicing is a non-reversible operation, effectively determining the estimated illuminant. Therefore any illuminant evaluation needs to happen beforehand. Another straightforward adaptation is to implement a more sophisticated demosaicing algorithm.

2.4 Colour Space Conversion

After demosaicing we obtained RGB values for each image point such that the resulting image can be displayed. However the pixel values are relative to the used sensor i.e. in a device dependent colour space. Viewing the image

directly results in a wrong interpretation of the colour values by the display device.

For correct display and printing we therefore have to convert the values into a standardized device independent colour space. We will use sRGB [10] as it is the most common one and the corresponding device specific colour conversion matrix is available in *imgdata.color.rgb_cam*

Listing 4:

```
cam2srgb = proc.imgdata.color.rgb_cam[:, 0:3]
cam2srgb = np.round(cam2srgb*255).astype(np.int16)
img = img // 2**6          # reduce dynamic range to 8bpp
shape = img.shape
pixels = img.reshape(-1, 3).T      # 3xN array of pixels
pixels = cam2srgb.dot(pixels)//255
img = pixels.T.reshape(shape)
img = np.clip(img, 0, 255).astype(np.uint8)
```

Here we convert the colour space conversion matrix to integer values and reduce the dynamic range of the image to 8bit to be able to continue with integer pixel values. To improve image quality a tone mapping operator can be used (see [4] for a survey). For HDR imaging one should convert the image to floating point instead.

2.5 Gamma Correction

We now have linear pixel values in the device independent sRGB colour space. However the sRGB standard also demands the values to be gamma corrected (gamma compressed) with $\gamma = 2.2$.

We perform gamma correction by first generating a look up table (LUT) to speed up the process. The gamma curve is only valid for values in the $[0, 1]$ range, therefore we scale the LUT to apply it to the given value range of $[0, 255]$.

Listing 5:

```
gcurve = [(i/255) ** (1/2.2) for i in range(256)]
gcurve = np.array(gcurve * 255, dtype=np.uint8)
img = gcurve[img]                  # apply gamma LUT
```

Note that we only use the approximate formula for gamma compression here instead of the piecewise function described in the sRGB standard.

3 Conclusions

The presented workflow allows loading RAW images and converting them to displayable RGB images. By only depending on Python it can be used and deployed as part of a larger image acquisition and processing pipeline — for instance in combination with OpenCV. All processing is done on integer types and superfluous copies are avoided which ensures good performance even without using C/C++. When used for batch processing of RAW files, no preprocessing by external tools is required, making the method self-contained and thus truly cross platform.

Still the described method only shows the minimal amount of processing for developing digital images. Further processing steps can be included considering additional metadata specified in the DNG standard. For instance image distortions were not corrected — even though the lens distortion parameters are present in the metadata (WarpRectilinear) and the according algorithm can be found in the OpenCV library (undistort).

Other sensible processing steps include noise reduction using the sensor noise profile, bad pixels elimination considering bad pixel locations and vignetting compensation by reading the aperture and the used lens.

References

[1] Adobe Systems Inc. Digital negative specification, 2012. URL `https://www.adobe.com/products/dng/pdfs/dng_spec_1_3_0_0.pdf`.

[2] G. Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.

[3] Dave Coffin. Decoding raw digital photos in linux, 2009. URL `http://www.cybercom.net/~dcoffin/dcraw/`. [Online; accessed 31-May-2015].

[4] Gabriel Eilertsen, Robert Wanat, Rafał K Mantiuk, and Jonas Unger. Evaluation of tone mapping operators for hdr-video. In *Computer Graphics Forum*, volume 32, pages 275–284. Wiley Online Library, 2013.

[5] Arjan Gijsenij, Theo Gevers, and Joost Van De Weijer. Computational color constancy: Survey and experiments. *IEEE Transactions on Image Processing*, 20(9):2475–2489, 2011.

[6] Tomomasa Gotoh and Masatoshi Okutomi. Direct super-resolution and registration using raw cfa images. In *Computer Vision and Pattern Recognition, 2004. CVPR 2004. Proceedings of the 2004 IEEE Computer Society Conference on*, volume 2, pages II–600. IEEE, 2004.

[7] Google Inc. Android API, 2015. URL `https://developer.android.com/reference/android/graphics/ImageFormat.html#RAW_SENSOR`. [Online; accessed 31-May-2015].

[8] Sang-Yong Lee and Antonio Ortega. A novel approach of image compression in digital cameras with a bayer color filter array. In *Image Processing, 2001. Proceedings. 2001 International Conference on*, volume 3, pages 482–485. IEEE, 2001.

[9] Xin Li, Bahadir Gunturk, and Lei Zhang. Image demosaicing: A systematic survey. In *Electronic Imaging 2008*, pages 68221J–68221J. International Society for Optics and Photonics, 2008.

[10] Michael Stokes, Matthew Anderson, Srinivasan Chandrasekar, and Ricardo Motta. A standard default color space for the internet — srgb, version 1.10, 1996. URL `http://www.w3.org/Graphics/Color/sRGB`.

[11] Rob Sumner. Processing raw images in matlab. 2014. URL `https://users.soe.ucsc.edu/~rcsumner/rawguide/`.

[12] Alex Tutubalin. Libraw, 2015. URL `https://github.com/LibRaw/LibRaw`. [Online; accessed 31-May-2015].

[13] Wikipedia. Color filter array — wikipedia, the free encyclopedia, 2015. URL `http://en.wikipedia.org/w/index.php?title=Color_filter_array&oldid=661805124`. [Online; accessed 31-May-2015].

[14] Wikipedia. Foveon x3 sensor — wikipedia, the free encyclopedia, 2015. URL `http://en.wikipedia.org/w/index.php?title=Foveon_X3_sensor&oldid=655552191`. [Online; accessed 7-June-2015].

[15] Wikipedia. Bayer filter — wikipedia, the free encyclopedia, 2016. URL https://en.wikipedia.org/w/index.php?title=Bayer_filter&oldid=713965003. [Online; accessed 30-May-2016].

[16] Lei Zhang, Rastislav Lukac, Xiaolin Wu, and David Zhang. Pca-based spatially adaptive denoising of cfa images for single-sensor digital cameras. *IEEE transactions on image processing*, 18(4):797–812, 2009.

[17] Xuemei Zhang and David H Brainard. Estimation of saturated pixel values in digital color imaging. *JOSA A*, 21(12):2301–2310, 2004.